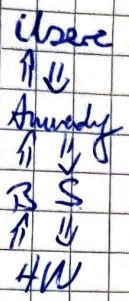
BS – Klausurrelevant

→ Betriebssystem:

Programme, die die Ressourcen eines Rechners verwalten und diese Anwendungsprogrammen zur Verfügung stellen.

Ziele:

→ Bequemlichkeit → Effizienz → Anpassbarkeit



- Betriebsystem:

Das BS ist eine Software, welche

→ alle Betriebsmittel eines Rechnersystems verwaltet

→ ihre Zuteilung koordiniert und

→ Nutzern des Rechnersystems eine VM anbietet

- Betriebsmittel

Alle zuteilbaren Hard- & Software-Komponenten eines Rechnersystems werden als Betriebsmittel bezeichnet.

- Virtuelle Maschine

Eine VM ist eine Abstraktion von der realen HW. Es ist Software, welche die Nutzung der realen HW über eine einfache Schnittstelle ermöglicht.

→ Aufgaben BS:

→ Ausführung von Anwendungsprogramm → Schutz gleichzeitig ausgeführter Programme

→ Datenaustausch mit Geräten → Mehrbenutzerbetrieb

→ Fehlererkennung & -meldungen → Speicherverwaltung

↝ BS-Kern (Kernel):

Zentraler Bestandteil eines BS, unterste Softwareschicht mit direktem Zugriff auf HW

↳ Benutzerschnittstellen ↳ Prozessmanagement
↳ Speichermanagement ↳ I/O Management
↳ Gerätetreiber
Systemcall-Interface

- **Lokalitätseigenschaft**

Die Lokalitätseigenschaft eines Prozesses besagt, dass die Wahrscheinlichkeit, dass Daten, die gerade benutzt werden, auch in naher Zukunft wieder benötigt werden, sehr hoch ist. Man unterscheidet zeitliche und räumliche Lokalität

- **Treiber**

Komponenten im Kernel, die Zugriff auf die Geräte-Register haben und Eigenarten der Geräte gegenüber anderen Komponenten verbergen.

↳ Zugriff auf die Peripherie ↳ Schnittstellen zur Einbindung von Treibern in Kern
↳ Gerätespezifische Schnittstellen
- Erlauben volle Ausnutzung aller Gerätefähigkeiten
- Erfordern Erweiterung des I/O-Systems für jeden Treiber

↳ Gerätenunabhängige Schnittstellen
- Uniforme I/O Schnittstelle: open(), read(), ...
- Garantieren einfache Erweiterbarkeit unabhängig von existierenden Anwendungen
- Erlauben virtuelle Geräte

BS - KUB

↳ Register des Prozessors

- Kontroll- und Status-Register:
  - Werden vom Prozessor selbst verwendet und dienen dem BS zur Kontrolle der Programmausführung
- Benutzersichtbare Register
  - ALU verarbeitet Daten in diesen Registern, Daten aus Hauptspeicher müssen zunächst in Register kopiert werden
  - Verwendung innerhalb von Maschinenprogrammen

- **Interrupt**

Ein Interrupt ist ein Signal, welches die normale Programmausführung unterbricht, um eine andere Verarbeitung durchzuführen.

↳ Verbessern Anweisungsdurchsatz von Prozessoren!

↳ Können durch Instruktionen (Trap) oder HW ausgelöst werden
- ↳ Vorhersagbar & reproduzierbar
- ↳ nicht reproduzierbar & vorhersagbar

- **Interrupt Handler, Interrupt Service Routine (ISR)**

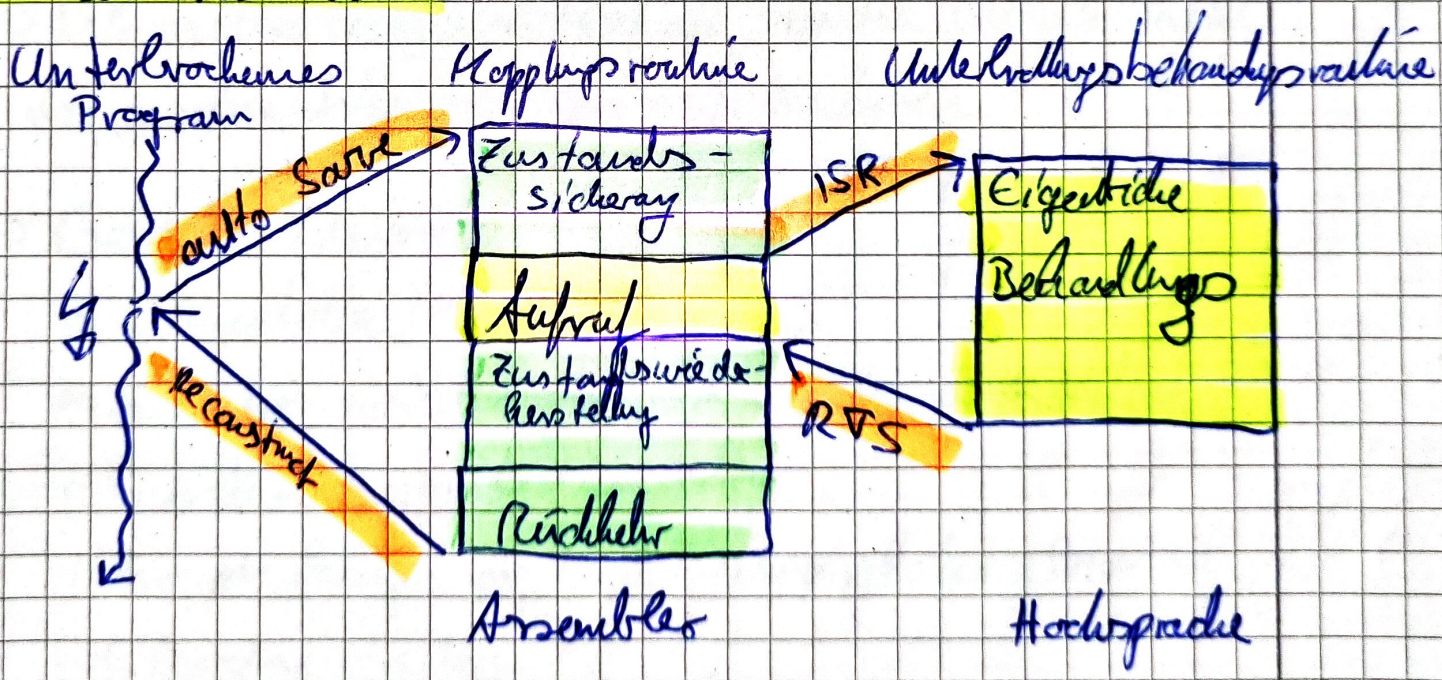
Programm, welches auf Grund eines Interrupts ein zugeordnetes Programm ausführt

↳ Bei Interrupt wird aktuelles Programm unterbrochen und ISR aufgerufen; Interruptvektortabelle ordnet jedem Interrupt-Typ die Startadresse einer ISR zu.

→ Interrupt Handler:

- Wiederaufnahme:
  Erfolgreiche Behandlung der Ausnahmesituation führt zur Fortsetzung der Ausführung des unterbrochenen Prozesses. Ein Trap kann, ein Interrupt muss nach diesem Modell behandelt werden
- Beendigung:
  Konnte die Ausnahmesituation nicht behandelt werden, wird ein schwerwiegender Fehler konstatiert, der zum Abbruch des unterbrochenen Prozesses führt. Ein Trap kann, ein Interrupt darf niemals nach diesem Modell behandelt werden.

→ Kontextwechsel



↳ Generierung der Maschinenbefehle durch Compiler
→ zur Sicherung / Wiederherstellung der Arbeitsregisterinhalte
→ zur Wiederaufnahme der Programmausführung

BS - HVB

↝ Interrupt Handler:

→ Jedem Interrupt wird eine Nummer zugeordnet, die als Index in einer Tabelle dient

→ Einträge der Vektortabelle führen zur ISR

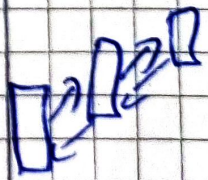
↝ Priorisierung von Interrupts:

- Softwarelösung: CPU hat nur einen Interrupt Eingang und fragt die Geräte in bestimmter Reihenfolge ab
- Hardwarelösung: Eine Priorisierungsschaltung ordnet Geräten eine Priorität zu und leitet immer nur die dringste Anforderung zur Behandlung weiter
  ↳ Feste Prioritäten ↳ Variable Prioritäten

↝ ISR sollten möglichst kurz sein, um die Wahrscheinlichkeit von gesperrten Interrupts zu minimieren

↝ Geschachtelte Interrupts

- Sequentielle Verarbeitung
  Verarbeitung von Interrupts wird in ISR abgeschaltet
- Prioritätengesteuerte Verarbeitung
  Interrupts können nur durch Interrupts höherer Priorität unterbrochen werden



BS - KVB

→ I/O Techniken:

1) Programmierte I/O
   - → I/O Modul führt Aktion durch
   - → Nach Ende setzt I/O Modul spezielles Register
   - → Prozessor prüft wiederholt Status dieses Register während des Lesevorgangs

   Nachteil: Zeitaufwendig & Prozessor wird unnötig beschäftigt

2) Interrupt-driven I/O
   - → Prozessor wird über Interrupt benachrichtigt
   - → Prozessor ist frei für andere Aktivitäten
   - → Nachteil: immer noch sehr prozessorintensive, da jedes Wort durch Prozessor muss

3) Direct Memory Access (DMA)
   - → I/O Datenaustausch erfolgt direkt mit Speicher
   - → Prozessor erlaubt I/O Modul den Zugriff auf den Speicher
   - → Prozessor wird nicht mehr für jedes Wort benötigt
   - → Ein Block wird direkt von externem Gerät in den Speicher kopiert
   - → Nach Ende der Aktion wird ein Interrupt gesetzt
   - → Prozessor ist während des ganzen Vorgangs frei für andere Aktionen
   - → Infos für DMA Modul: ⇝ Art der Operation
     ⇝ Adresse des I/O Gerätes
     ⇝ Anzahl der zu schreibenen Worte

CPU → DMA

Issue read block command to I/O module ---→ Do something else

Read Status of DMA module ← Interrupt

DMA → CPU

BS - VL 03

- **Booten, Bootstrapping**

  Booten (Hochfahren) bezeichnet das Laden des BS eines Computers. Nach dem Boot-Vorgang ist der Rechner betriebsbereit.

  Bootstrapping bezeichnet in der IT einen Prozess, der auf einem einfachen System ein komplexes System aktiviert.

→ BIOS = Basic Input/Output System (Firmware)

↳ Das BIOS überprüft zuerst, ob alle zum Start benötigten Soft- & Hardware-Komponenten einsatzbereit sind und lädt dann das BS in den RAM-Speicher

BS
BIOS
HW

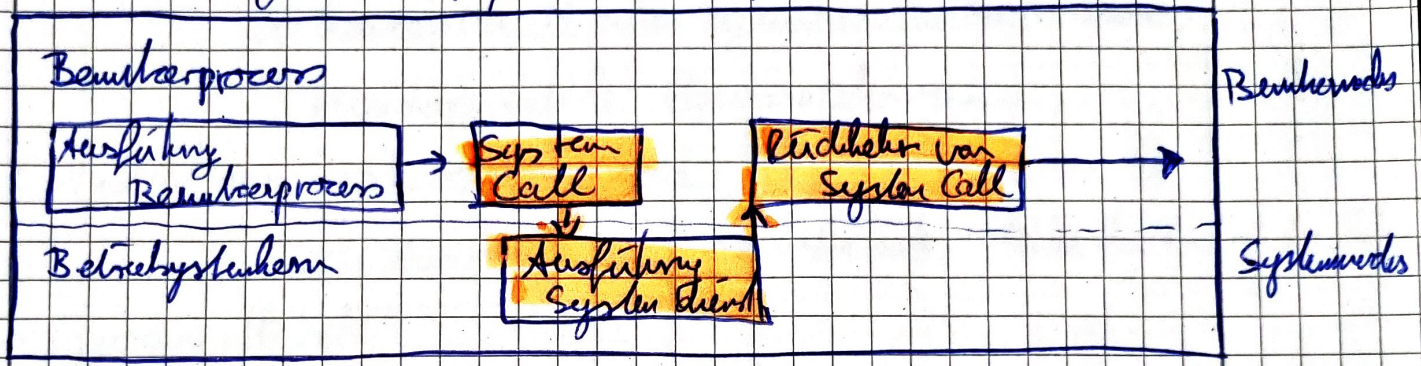
→ User Modus & Kernel Modus

- Nur im Kernel dürfen alle Operationen ausgeführt werden
  - ↳ Zugriff auf HW
  - ↳ Erzeugung neuer Prozesse
  - ↳ Manipulation von systemrelevanten Datenstrukturen
  - ↳ Zugriff auf Register
- Anwendungen werden im User-Modus ausgeführt

BS - UMB

- **Systemaufruf (system call)**
  Ein Systemaufruf ist eine Methode, um alle vom BS bereitgestellten Funktionalitäten auszuführen. Dabei wird die Kontrolle vom Programm an den Kernel übergeben.

↳ Schnittstelle zu Anwendungsprogrammen
↳ Überwachung von Zugriffsrechten auf Dateien und Geräte



↝ mode bit steuert Übergang in Kernel-Modus
↝ mode bit kann nur durch Systemaufruf verändert werden

User Mode: nur ausgewählte Maschinenbefehle, HW-Zugriff nur über BS, kein/nur lesender Zugriff auf Systemdaten
Kernel-Modus: Alle Maschinenbefehle, Vollzugriff auf HW, Vollzugriff auf Systemdaten

Ablauf: ↝ Systemcall lösen Eintritt in Kernel Modus aus:
↳ Benutzerprogramm kopiert Parameter & ID der Funktion in Hauptspeicher und erzeugt Trap
↳ Prozessor unterbricht Anwendung, startet Trap-Handler
↳ Übergabe CPU-Kontrolle an BS, mode bit = 1
↳ CPU arbeitet Aufruf ab. Am Ende kopiert aufgerufene Funktion Resultat in Speicher Bereich der Anwendung ↳ Trap wird abgearbeitet, Rückkehr des Prozess mode bit = 0

BS - UVO

- Prozess

Ein in Ausführung befindliches Programm, einschließlich der aktuellen Werte von Register, Programmzähler, Stack, Speichervariablen.

↳ besitzt: → einen privaten Adressraum, der den Teil des Hauptspeichers vereinbart, welchen der Prozess nutzen kann
→ konzeptionell einen eigenen virtuellen Prozessor, pseudoparallele Prozessausführung

↳ besteht aus: → Programm Code
→ Zustand → Attribute, welche den Zustand beschreiben

⇝ Prozessattribute:
Prozessbezeichner (PID), Ablaufzustand, Priorität, Speicherverweise, Envs, I/O Statusinformation, Verwaltungsinformationen

- Dispatcher / Scheduler

Der Dispatcher ist ein Programm, welches die Umschaltung zwischen Prozessen realisiert.
Der Scheduler ist ein Programm, welches bestimmt, wann Prozesse unterbrochen werden und zu welchen Prozess gewechselt wird

↳ Sicht des Prozesses; Prozess wird immer ausgeführt!

⇒ CPU Auslastung (Grundsicht)

$$= \frac{\text{Prozess Zeit}}{\text{Gesamt Zeit}} = \frac{6}{6+6} = 0{,}50 \hat{=} 50\,\%$$

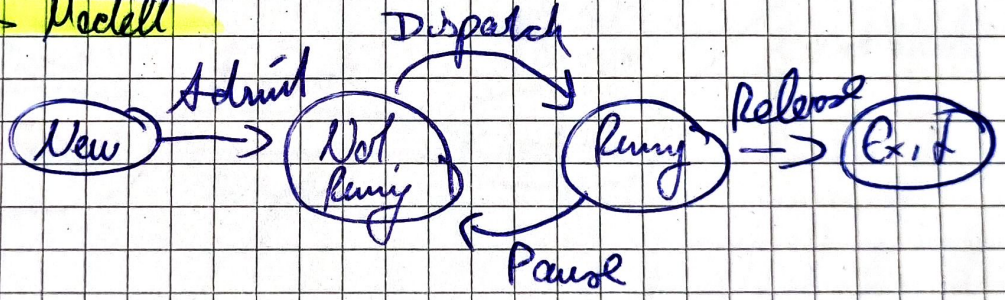
BS - HVB

→ Prozess vs Programm:

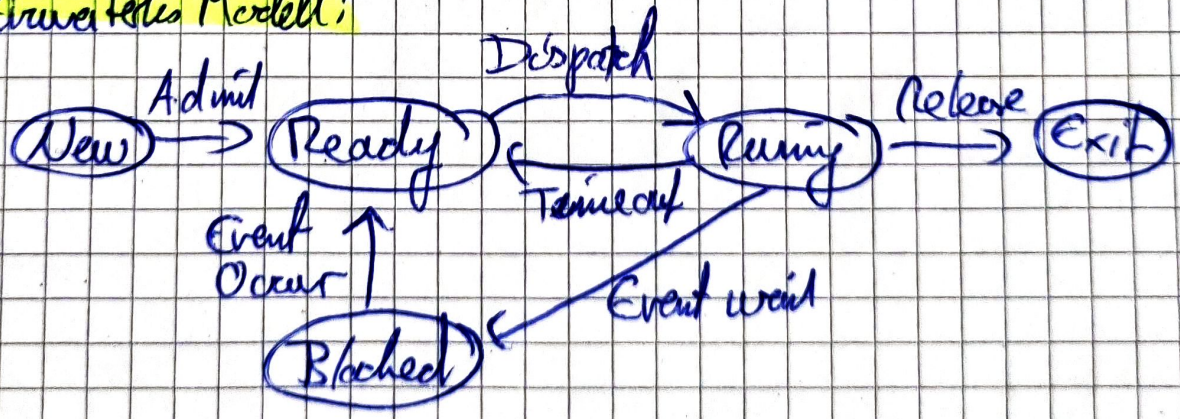
- Programm: → statische Beschreibung eines Algorithmus
  → Ein Programm kann zur gleichen Zeit durch verschiedene Prozesse ausgeführt werden
- Prozess: → Ausführungsgeschwindigkeit eines Prozesses nicht gleichmäßig, nicht reproduzierbar
  → Programmierer können keine Annahme über zeitlichen Verlauf machen

- Prozesszustandsmodell:
  Ein Prozesszustandsmodell beschreibt die möglichen Zustände eines Prozesses und alle möglichen Übergänge zwischen Zuständen

→ Einfaches Modell



→ Erweitertes Modell:



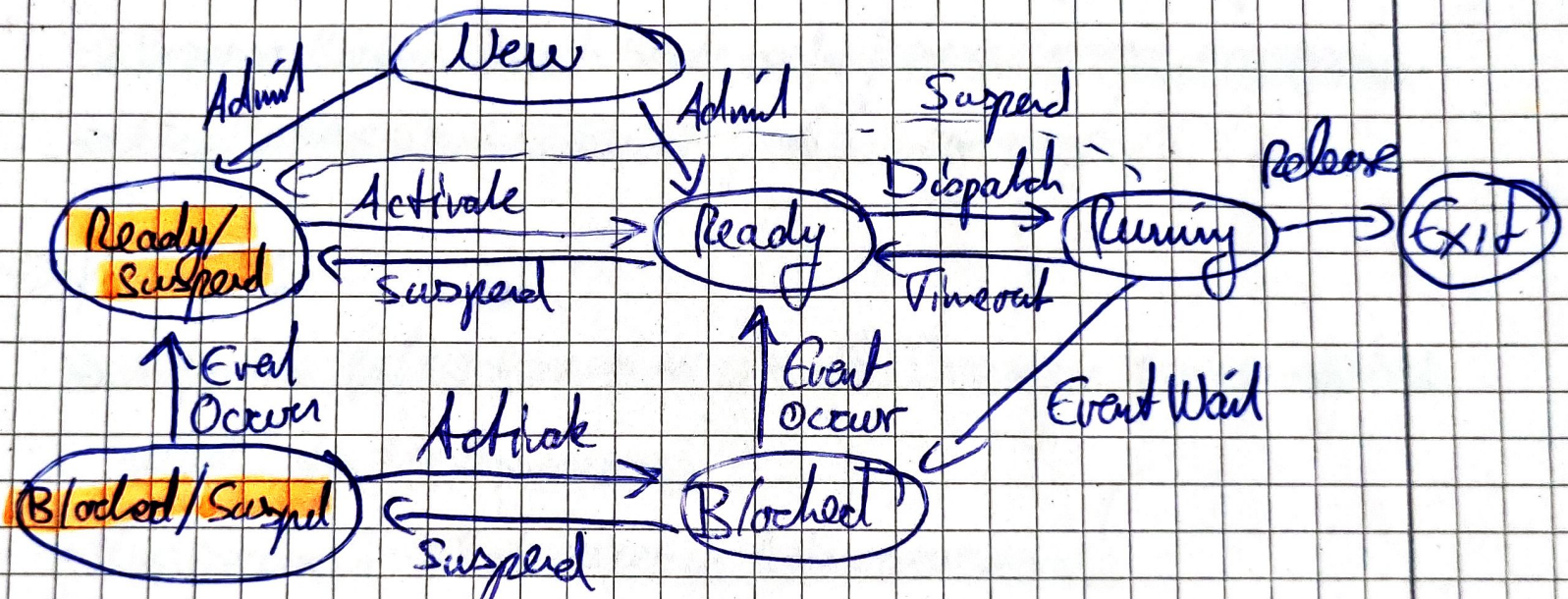
BS - KVB

- **Swapping / Auslagern**

  Das Auslagern der Daten eines blockierten Prozesses auf persistenten Speicher wird als Swapping bezeichnet.

↳ durch swapping wird Hauptspeicher für andere Prozesse frei

↳ Gründe: → Freigabe von Hauptspeicher für anderen Prozess
→ Prozess, der Probleme verursacht, wird suspendiert
→ Benutzerwunsch



BS - 11/13

~> Prozessterminierung:

- freiwillig: Programmende oder Aufruf von Exit()
- unfreiwillig: ~> Zeitüberschreitung überschritten
  ~> Nicht ausreichend Speicher zur Verfügung
  ~> arithmetischer Fehler
  ~> I/O - Fehler

~> Prozesserzeugung:

-> Beim Booten des Systems wird ein Initialprozess gestartet
-> Prozesse können durch Systemaufruf neue Prozesse erzeugen
-> Alle Prozesse werden durch fork() erzeugt

~> fork():

-> Aufruf von fork() erzeugt der aktuelle Prozess eine Kopie von sich selbst, den Kindprozess

- Kindprozess: -> hat eigenen Adressraum
  -> übernimmt Daten, Code & PC von Elternprozess
  -> erhält eigene PID

↳ BS führt Child unabhängig von Parent aus

-> Rückgabe wird von fork()
  -> 0 bedeutet Child
  -> > 0 bedeutet Parent (Child-ID)
  -> -1 Fehler

BS-ÜVB

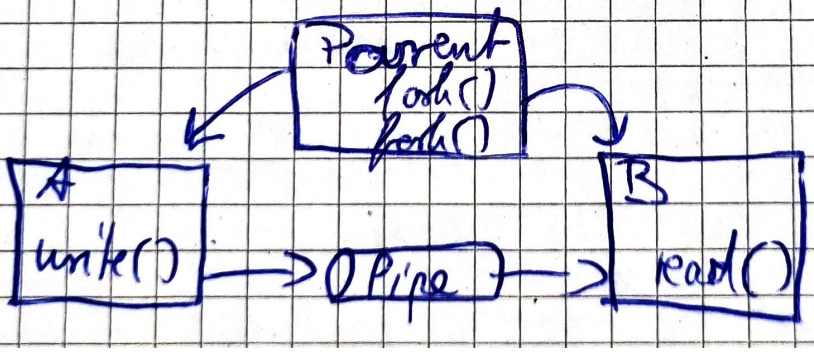
→ fork & exec Mechanismus

- exec(), dann wird das ursprüngliche Programm durch ein neues überschrieben und mit dessen Ablauf begonnen.
  - ↳ Argumente von exec(): Pfadangabe des zu ladenen Programms
  - ↳ Umgebung des Prozesses bleibt erhalten
  - ↳ Ist exec() Aufruf erfolgreich, dann kehrt dieser nicht zurück, bei einem Fehler liefert dieser -1 zurück

→ Interprozesskommunikation

- **Pipe**

Eine Pipe ist ein Ringpuffer zur Unterstützung der Kommunikation von zwei Prozessen.



BS – KVB

- **Signal**

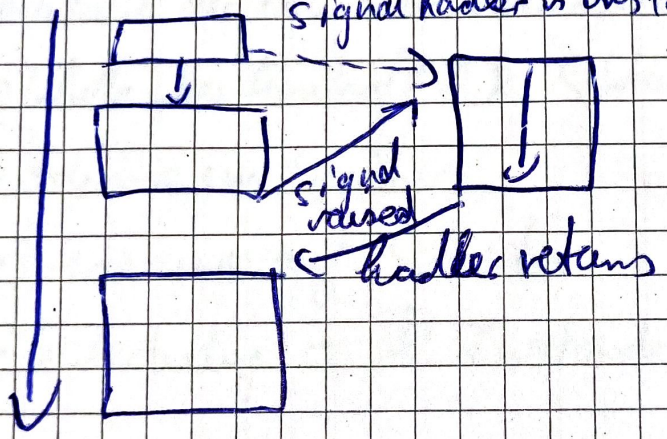
Ein Signal ist eine Systemnachricht an einen laufenden Prozess.

↳ Asynchrone Interprozesskommunikation

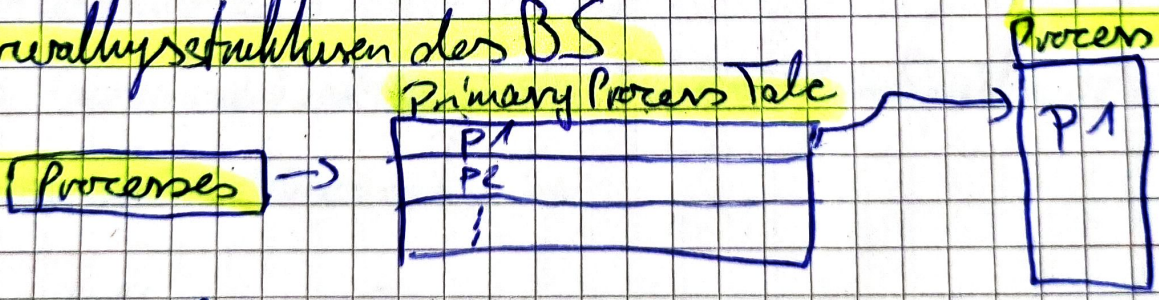
↳ Es gibt keine Anweisung zur Angabe des Empfangsbereitschaft

↳ Beim Auftreten eines Signals wird Prozess unterbrochen und eine vom Prozess festgelegte Signal-Behandlungsroutine aufgerufen

↳ Prozess kann Signale ignorieren



→ Verwaltungsstrukturen des BS



↳ Speichertabelle & I/O-Tabelle

BS - VL 6

→ Speicherverwaltung: → Programmspeicher
→ Speicher für globale & statische lokale Variablen
→ Heap → Stack

- Prozesskontrollblock (PCB)

Der PCB fasst alle zu einem einzelnen Prozess gehörenden Informationen zusammen

↳ Inhalt:
- Prozesserkennung: PID, PPID, UID
- Inhalte der benutzersichtbaren Register
- Inhalte der Kontroll- & Status-Register: PC
- Adresse im Speicher
- Verweise auf Stacks
- Scheduling- & Zustandsinformationen

- Prozessimage

Die Gesamtdarstellung eines Prozesses im Speicher wird als Prozessimage bezeichnet.

↳ Inhalt: → PCB
→ Benutzerdaten
→ Programm
→ Stacks

BS - HVB

→ Prozesswechsel:

1) Prozess anhalten
2) Wechsel im Kernel-Modus
3) PCB aktualisieren (Zustand speichern)
4) Nächsten Prozess auswählen
5) MMU für nächsten Prozess vorbereiten
6) PCB des nächsten Prozesses aktualisieren und dessen Zustand laden
7) Wechsel in User-Modus
8) Ausführung der nächsten Anweisung

Wann?: BS veranlasst Wechsel, Prozess beendet sich, Prozess blockiert

→ Prozesswechsel vs Kontextwechsel

Wann findet kein Prozesswechsel statt?

↳ Bei einem Systemaufruf
↳ Bei Wechsel in den Kernel Modus } Kontextwechsel
↳ Interrupt

BS-NVB-Ex1

(1-1)

1) Was ist ein Prozess?

Ein Prozess ist ein in Ausführung befindliches Programm, einschließlich der aktuellen Werte von Registern, PC, Stack, Speichervariablen

→ Ausführungsgeschwindigkeit eines Prozesses nicht gleichmäßig!

2) Wodurch wird ein Prozess beschrieben?

Ein Prozess besteht aus: Programm-Code, Zustand, Attributen

3) Wodurch wird ein Prozess im BS repräsentiert?

Durch den PCB.

Der PCB fasst alle zu einem einzelnen Prozess gehörenden Informationen zusammen.

Inhalt: Prozessverwaltung (PID, PPID, UID),
Inhalte der benutzersichtbaren Register,
Inhalte der Kontroll- & Status-Register (PC, etc.),
Adresse im Speicher,
Verweise auf Stacks,
Scheduling- & Zustandsinformationen

BS-HÜB-Ex1

(1-1)

4) Wie werden Prozesse erzeugt bzw Programme gestartet?

→ Beim Booten des Systems wird Initialprozess gestartet
→ Alle weiteren Prozesse werden durch fork() erzeugt
→ fork() erzeugt eine Kopie von einem Prozess und mittels exec() wird ein Neues Programm geladen und ausgeführt.

5) Was passiert während der Prozesserzeugung?
Beim Aufruf von fork() erzeugt der aktuelle Prozess eine Kopie von sich selbst, den Kindprozess

↳ Kindprozess: → hat eigenen Adressraum
→ übernimmt Daten, Code & PC vom Parent
→ erhält PID

6) Wie kann ein Zombie-Prozess entstehen?

Zombie:
Process no longer exists, but it leaves a record for its parent process to collect.
(Child processes end before parent process becomes Zombie)

BS - KVB - Ex 1

1-1

a) Dispatcher & Scheduler?

Der Dispatcher ist ein Programm, welches die Umschaltung zwischen Prozessen realisiert.

Der Scheduler ist ein Programm, welches bestimmt, wann Prozesse unterbrochen werden und zu welchem Prozess gewechselt wird.

b) Wozu dient die Interprozesskommunikation?

→ Austausch Informationen zwischen kooperierenden Prozessen

→ Synchronisation kooperierender Prozesse

BS - WUB - Ex 1

(1-2) fork()

Programm siehe Ex 1.

waitpid() wartet auf das Beenden des Prozesses mit der Prozess-ID, die in der Variable childPid gespeichert ist.

a) Was gibt dieses Programm aus & warum?

2 1 ← I hat unterschiedliche Werte da I aus unterschiedlichen Speichern

↑ Reihenfolge durch waitpid vorbestimmt!

b) Wenn waitpid entfernt wird?

2 1 / 1 2 Reihenfolge der Ausgabe nicht mehr vorbestimmt ⇒ undeterministisch

c) Was passiert mit dem Child, wenn, der Kindprozess auf eine Eingabe wartet und der Parent sich schon beendet?

↳ Es entsteht ein Waisenprozess ⇒ PPID = 1

BS-HÜB - Ex1

(1-3) Shell

Siehe Ex1.

Bei jedem Programmstart erzeugt die Shell immer zuerst eine Kopie von sich selbst, die dann danach direkt mit dem neuen Programm überladen wird. Da das Erzeugen einer Kopie viel Zeit kostet, ist dieser Vorgang eine Verschwendung. Welche Möglichkeit hat das BS, diese Verschwendung zu vermeiden?

→ vfork(): Erzeugt einen neuen Prozess und der Parent wird suspendiert, bis der Child sich beendet oder exec() aufruft. Bis dahin darf der Child keine Daten verändern.

→ copy-on-write: Dabei teilen sich Parent & Child den gleichen Speicher (kein Kopieren). Erst bei Änderungen wird das entsprechende Speichersegment kopiert. Daher wird nur der PCB & etwas Speicher bereits der Prozesserzeugung kopiert.

BS - KVB - Ex1

(1-4) Prozessumgebung

```
ls > output.txt // ls wird ausgeführt und output wird
                   mittels Pipe zum output.txt file übergeben

wc -l < output.txt   // wc -l bedeutet als Std/in
                        das file

ls | wc -l // ls output wird zu wc -l gepiped
              als eingabe
```

BS – GNB-Praxis 1 & 2

int execlp(const char *file, const char *arg, ...)
↳ Das laufende Programm wird mit dem angegebenen Program überlagert und dieses wird dann mit den übergebenen Parametern ausgeführt.

PATH
↳ Eine Liste von Verzeichnissen, in denen nach Programmen gesucht wird

int pipe(pipefd[2])
↳ Erstellt eine Unnamed Pipe.

child: dup2(pipe[1], STDOUT)
Parent: dup2(pipe[0], STDIN)
child ⇒ Parent

- Kann ein Prozess für sich feststellen, dass er ein Waisen-Prozess ist?
  if getppid() == 1 ⇒ Orphan!

- Zombie Prozess?
  ⇝ Normale Situation ⇝ zusätzlich = Z

- Nachgelernt eines Prozesses?
  Child: execl(PATH_TO_DICE)
  Parent: int status
  waitpid(childID, &status, 0);
  WEXITSTATUS(status); // Gibt Exit Status von Child

- Umleitung stdout:
  fd = fopen(...)
  dup2(fileno(fd), STDOUT) // Sage alles was zu STDOUT soll wird an fd